Software: Practice and Experience

WILEY

Allergy EUROPEAN JOURNAL OF ALLERGY AND CLINICAL IMMUNOLOGY EAACI

Original Article OPEN ACCESS

Paper Agent: A Local-First Human-in-the-Loop Agentic Workspace for Controllable Academic Writing

ZuKang Xu

[1]Houmo AI, Jiangsu, China |

Correspondence: ZuKang Xu, zukang.xu@houmo.ai

Keywords: academic writing large language models human-in-the-loop document engineering LaTeXsoftware architecture agentic workflow

ABSTRACT

Large language models (LLMs) are increasingly used to assist academic writing, yet general-purpose chat interfaces disconnect model output from the scholarly artifacts—LaTeX source files, bibliographic databases, figures, compiler logs, and reviewer feedback—that constitute a real manuscript. We present *Paper Agent*, a local-first, human-in-the-loop software platform that treats each manuscript as a filesystem-backed project and integrates AI assistance into a complete document engineering workflow. The system combines a React/Vite frontend with a Fastify/Node.js backend and provides permission-aware AI interaction modes (Chat, Agent, Tools), typed workflow pipelines with human checkpoints, citation verification against scholarly databases (CrossRef, Semantic Scholar, OpenAlex), LaTeX compilation via multiple engines, Model Context Protocol (MCP) interoperability, and an integrated tmux terminal. We describe the design rationale, architecture, and implementation of *Paper Agent*, report on practical experience from developing and using the system, and discuss lessons learned about building controllable AI-assisted writing environments. The implementation comprises approximately 10,600 lines of backend JavaScript and 12,500 lines of frontend TypeScript/JSX, validated by a 32-file Vitest suite containing 227 automated tests plus a Playwright end-to-end scenario. We argue that treating manuscript writing as a verifiable document engineering pipeline—rather than an isolated text-generation task—enables researchers to retain control over AI-generated edits while benefiting from language model assistance.

1 | Introduction

Academic writing is a complex document engineering process. In disciplines that use LaTeX, a manuscript is not merely a linear text document: it is a project composed of source files, bibliographic databases, figures, style templates, experimental code, compiler logs, reviewer feedback, and final PDF artifacts. Researchers must coordinate editing, citation management, compilation, revision tracking, and collaboration across these interconnected components.

The emergence of large language models (LLMs) [25, 1, 24] has introduced new possibilities for AI-assisted writing. Tools such as ChatGPT, Claude, Gemini, and specialized academic assistants can generate, paraphrase, summarize, and critique scholarly text. However, general-purpose chat interfaces treat writing as an isolated text-generation task. They separate model output from the project context in which scholarly artifacts must be edited, cited, compiled, and reviewed. This separation creates three practical problems:

1. **Context fragmentation.** Researchers must repeatedly copy manuscript sections into the assistant, increasing friction and the risk of stale or incomplete prompts. When the manuscript changes, chat context diverges from source truth.

2. **Unaudited edits.** When assistants can directly modify files, or when changes are manually copied into a source tree, it becomes difficult to track what was changed, by whom, and why. This undermines the audit trail essential for scholarly integrity.
3. **Unverifiable artifacts.** Generated text may introduce hallucinated references, missing citation keys, or LaTeX changes that fail to compile. Liang et al. [19] documented widespread use of LLMs in scientific manuscripts and highlighted significant risks of hallucinated references, underscoring the verification gap. These are document engineering failures that text-generation quality metrics do not capture.

Existing solutions address parts of this problem—Overleaf [27] provides collaborative LaTeX editing with recently introduced AI features [28]; citation managers such as Zotero and Mendeley handle references; and AI chat interfaces generate text. Yet no existing system integrates these capabilities into a unified, controllable, local-first workspace designed for the full manuscript lifecycle.

We present *Paper Agent*, a local-first, human-in-the-loop software platform that addresses this gap by treating academic writing as a software-backed document engineering workflow. The system is designed and implemented following practical software engineering principles: a modular monorepo architecture, comprehensive automated testing across 32 test files, secure configuration management with masked credentials, and careful attention to real-world deployment concerns such as API rate limiting, error recovery, and dependency management. The system represents each paper as a filesystem-backed project and provides an integrated workspace for editing, preview, compilation, citation verification, AI assistance, human checkpoints, and workflow orchestration. AI interaction is separated into three permission-aware modes to give researchers fine-grained control over how and when model output enters their manuscript.

This paper makes the following contributions:

- **System architecture.** We describe the design and architecture of *Paper Agent*, a local-first platform integrating AI-assisted writing with project-level document management, citation verification against multiple scholarly databases, LaTeX compilation, and tmux-based terminal management. The implementation comprises approximately 23,100 lines of application source code across a monorepo with React/Vite frontend and Fastify/Node.js backend.
- **Permission-aware interaction model.** We present a three-mode AI interaction model (Chat, Agent, Tools) that separates read-only discussion, reviewable edit proposals with inline diff display, and controlled tool execution. This design emerged from iterative user feedback during development and is designed to reduce anxiety about unintended manuscript changes and promote exploratory AI use.
- **Typed workflow pipelines.** We report on Pipeline 2.0, a system of typed pipeline stages (AI, Compute, Human, Citation, Compile) with human checkpoints as first-class execution gates, enabling auditable multi-stage writing processes.
- **Practical experience.** We share lessons learned from building and using the system across multiple development iterations, including design tradeoffs, testing strategies, the challenges of LLM integration, and validation through a 32-file automated test suite.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system design and architecture. Section 4 describes implementation details. Section 5 reports evaluation results and practical experience. Section 6 discusses lessons learned. Section 7 outlines limitations, and Section 8 concludes.

2 | Related Work

2.1 | AI-Assisted Academic Writing

Recent advances in large language models [25, 1] have spurred interest in AI-assisted scholarly writing. General-purpose chat interfaces such as ChatGPT, Claude, and Gemini provide text generation capabilities but lack project-level awareness of manuscript structure, citation databases, and compilation requirements.

Specialized tools address specific aspects of the writing process. Writefull [42] provides language polishing and grammar checking for academic text through a dedicated interface. Paperpal [3] offers comprehensive manuscript preparation assistance, including language checks and reference formatting. Trinka [38] specializes in technical and academic language correction. However, these tools operate on individual passages rather than managing complete manuscript projects with source files, bibliographies, and compilation.

Research on LLM-assisted writing has explored prompt engineering for academic genres, fine-tuning for scientific domains, and evaluation of generated text quality. Liang et al. [19] conducted a systematic survey of LLM use across the

academic writing pipeline, identifying tasks ranging from ideation to revision. They found that while LLMs are widely used for drafting and editing, systematic integration with document engineering infrastructure remains limited. These studies evaluate text quality in isolation rather than the end-to-end workflow of producing compilable, verifiable scholarly documents—a gap that *Paper Agent* aims to fill. A further challenge is that preprint repositories such as arXiv assign DOIs (`10.48550/arXiv....`) that CrossRef does not index, requiring dedicated API integration that current writing tools do not provide.

2.2 | Collaborative LaTeX Editing

Online LaTeX editing platforms dominate collaborative scholarly writing. Overleaf [27] provides real-time collaboration, version history, and integrated compilation, serving over 10 million users globally. Overleaf has recently introduced AI-assisted features [28], including grammar suggestions and summarization. Yet its cloud-centric architecture means manuscripts reside on remote servers, and AI integration is constrained by the platform's service model. Researchers concerned about data privacy or requiring offline access must seek alternatives.

Local LaTeX editors such as TeXstudio [37], VS Code with the LaTeX Workshop extension, and Emacs with AUCTeX provide powerful editing environments with syntax highlighting, code completion, and local compilation. While cloud platforms offer real-time collaboration out of the box, local setups provide superior compilation performance, offline availability, and full control over TeX distributions—at the cost of manual environment configuration. These tools increasingly support AI integration through extensions, but none provide a unified, project-aware AI workspace with built-in verification and workflow orchestration.

Modern reproducible-authoring tools address a related but distinct problem. R Markdown [30] and Quarto [29] combine prose, executable code cells, citations, and multi-format rendering, making them strong choices for computational notebooks, statistical reports, and literate data analysis. Their primary abstraction is the executable document: code and narrative are woven into HTML, PDF, Word, or presentation outputs. Quarto, the successor to R Markdown, provides a unified authoring format that targets multiple output engines—LaTeX, Typst, and revealjs—and integrates with version control, project-level metadata (`_quarto.yml`), and CI/CD pipelines. RStudio (also from Posit, the Quarto developer) provides an IDE that tightly couples code execution, notebook preview, and package management for R and Python workflows.

Paper Agent differs from Quarto and RStudio Markdown in three fundamental ways. First, the primary artifact: Quarto and R Markdown treat the `.qmd`/`.Rmd` source file as the unit of authoring, while *Paper Agent* treats the manuscript directory itself as the primary artifact, preserving ordinary LaTeX project structure with separate `.tex`, `.bib`, and `.sty` files. Second, the role of AI: Quarto and RStudio do not provide built-in AI assistance with permission-aware interaction modes; AI integration in these ecosystems typically relies on external extensions (e.g., GitHub Copilot in RStudio) that operate at the text-buffer level without project-level awareness of citation databases or compilation state. *Paper Agent* places AI assistance behind explicit permission modes (Chat, Agent, Tools) and integrates it with project-level services such as citation verification and compilation. Third, the verification model: Quarto focuses on reproducible rendering from code chunks, while *Paper Agent* focuses on verifying that AI-generated edits preserve compilability, citation accuracy, and document integrity. Thus, Quarto and R Markdown emphasize reproducible publishing from code-rich documents, whereas *Paper Agent* emphasizes controlled AI assistance, citation verification, and project-level document engineering for existing LaTeX manuscript workflows. These approaches are complementary rather than competing: a researcher could use Quarto for computational reproducibility and *Paper Agent* for AI-assisted manuscript engineering within the same project.

2.3 | Agent-Based Writing Systems

The concept of AI agents—systems that can plan, use tools, and execute multi-step tasks—has been applied to writing. Microsoft's Research Copilot [22] and related tools augment research workflows with AI capabilities, including literature review, data analysis, and writing assistance. LangChain [17], AutoGen [43], CrewAI [4], and LangGraph [18] enable tool-using or multi-agent workflows where different agents handle planning, drafting, reviewing, and execution.

However, these systems typically operate on plain text and lack direct integration with LaTeX project structures, citation databases, and compilation pipelines. Their abstractions are general-purpose chains, agents, tools, and conversations; the manuscript-specific responsibilities—preserving cross-references, checking BibTeX keys, compiling LaTeX, and deciding whether an edit should enter the source tree—must be assembled by the application developer.

LangChain provides composable chains and agent executors that can orchestrate multi-step LLM interactions with tool use. In an academic writing scenario, one could build a LangChain agent that drafts sections, verifies citations via custom tools, and compiles LaTeX documents. However, LangChain's default agent loop applies tool outputs automatically

and continues execution without mandatory human approval gates. The researcher becomes a post-hoc reviewer of fully generated output rather than an active decision-maker at each step. AutoGen extends this model by enabling multi-agent conversations where different agents (e.g., a "writer," a "reviewer," an "editor") collaborate autonomously. While this architecture can produce sophisticated outputs through agent deliberation, it further reduces the human role: the researcher observes agent-to-agent dialogue and intervenes only when the autonomous loop completes or stalls. CrewAI adopts a role-based orchestration pattern where agents are assigned specialized roles and goals, executing tasks in sequence or parallel. All three frameworks share a common design philosophy: AI as an autonomous executor that minimizes human intervention, with the human serving primarily as a supervisor or approver of final results.

Paper Agent takes the opposite approach: *AI as a proposer rather than a replacement author.* Chat mode cannot modify files, Agent mode returns reviewable diffs that require explicit accept/reject actions, and Tools mode is explicitly scoped to the project directory with full audit logging. This "proposer-not-replacer" philosophy differs fundamentally from the "autonomous executor" paradigm of LangChain, AutoGen, and CrewAI in three ways: (1) every file modification requires a deliberate human action; (2) the system never auto-applies AI-generated changes and never continues execution past a human checkpoint without explicit approval; and (3) the audit trail records not only what the AI proposed but also whether the human accepted or rejected each change. This design keeps scholarly authorship and final editorial authority with the researcher while still allowing agentic tooling to assist with bounded document engineering tasks. The tradeoff is reduced automation: tasks that LangChain or AutoGen could complete autonomously (e.g., "revise all sections for clarity and recompile") require human interaction at each checkpoint in *Paper Agent*. We argue that this tradeoff is appropriate for scholarly writing, where authorial accountability cannot be delegated to an autonomous agent.

2.4 | Human-in-the-Loop AI Systems

Human-in-the-loop (HITL) approaches have been extensively studied in machine learning [44], where human feedback guides model training and decision-making. In writing tools, HITL principles suggest that AI should propose rather than impose changes, and that humans should retain decision authority at key junctures.

Grammarly [10] exemplifies HITL in writing tools: it highlights issues and suggests changes, but the user applies them manually. GitHub Copilot [8] applies similar principles for code generation—it shows completions in-line, and the developer decides whether to accept. *Paper Agent* generalizes this principle across the entire academic writing workflow: the Chat mode enables free-form discussion without risk of changes, the Agent mode shows proposed edits as inline diffs for review, and the Tools mode provides controlled execution for multi-step operations within a sandboxed directory.

2.5 | Software Tools for Research Software Engineering

Software: Practice and Experience has a tradition of publishing papers on research software tools and platforms that emphasize practical engineering contributions. Notable examples include Jupyter [16] for interactive computing, the Git ecosystem for version control [33], and continuous integration platforms for research software [9]. These contributions share a common emphasis on practical software engineering: tools designed for real workflows, validated through use, and documented with attention to implementation tradeoffs.

Paper Agent contributes to this tradition by providing a practical software platform for AI-assisted scholarly writing, with emphasis on real-world design decisions, implementation experience, and lessons learned from building a system that bridges LLMs and document engineering. The paper follows the SPE convention of reporting on an implemented system with attention to architecture, testing methodology, performance characteristics, and deployment considerations—rather than positioning the contribution primarily as a theoretical advance or a user study.

3 | System Design and Architecture

3.1 | Design Goals

Paper Agent was designed to satisfy six key requirements distilled from the experience of the authors and feedback from early users in the research software community:

RG1: Local-first project model. Manuscript content must reside on the researcher's local filesystem as ordinary directories and files, compatible with version control (git), external tools (grep, diff), and manual inspection. The system should augment—not replace—existing file-based workflows.

RG2: Permission-aware AI interaction. AI assistance must be separated into distinguishable modes with clear boundaries: read-only discussion, reviewable edit proposals with inline diff visualization, and controlled tool execution. Users must be able to inspect and approve every file change before it is applied.

RG3: Verifiable artifact production. The system must integrate citation verification and LaTeX compilation so that generated manuscripts can be checked for reference accuracy and build correctness, not just text fluency.

RG4: Auditable workflow pipelines. Multi-stage writing workflows must be executable with human checkpoints at decision points, producing a record of what AI stages were executed, what human feedback was provided, and what artifacts were produced at each stage.

RG5: Interoperability. The system must expose core capabilities through standard protocols so that external tools and MCP-compatible clients can integrate with the platform.

RG6: Privacy and configuration safety. API keys and sensitive configuration must be stored in local environment files, never exposed to the browser, and masked in API responses.

Figure 1 highlights the implemented workspace features that distinguish *Paper Agent* from a general-purpose editor or chatbot. The left pane keeps the filesystem-backed project visible as the source of truth, the center pane couples LaTeX source editing with compiled PDF feedback, and the right pane exposes AI assistance and typed pipeline controls inside the same local project context.

3.2 | Architecture Overview

Paper Agent follows a client-server architecture with a filesystem-backed project model. Figure 2 provides a high-level overview of the system components and their interactions.

The system comprises four main layers:

Presentation layer (React 18 / Vite / TypeScript). A three-panel browser interface provides a project file tree, a multi-mode editor (source, split, rendered) with CodeMirror 6, LaTeX preview via KaTeX, PDF viewing via the browser's native PDF viewer, and a right panel with tabs for AI Chat, Skills (40+ writing plugins), Review, Anti-AI detection, and Pipeline 2.0 controls.

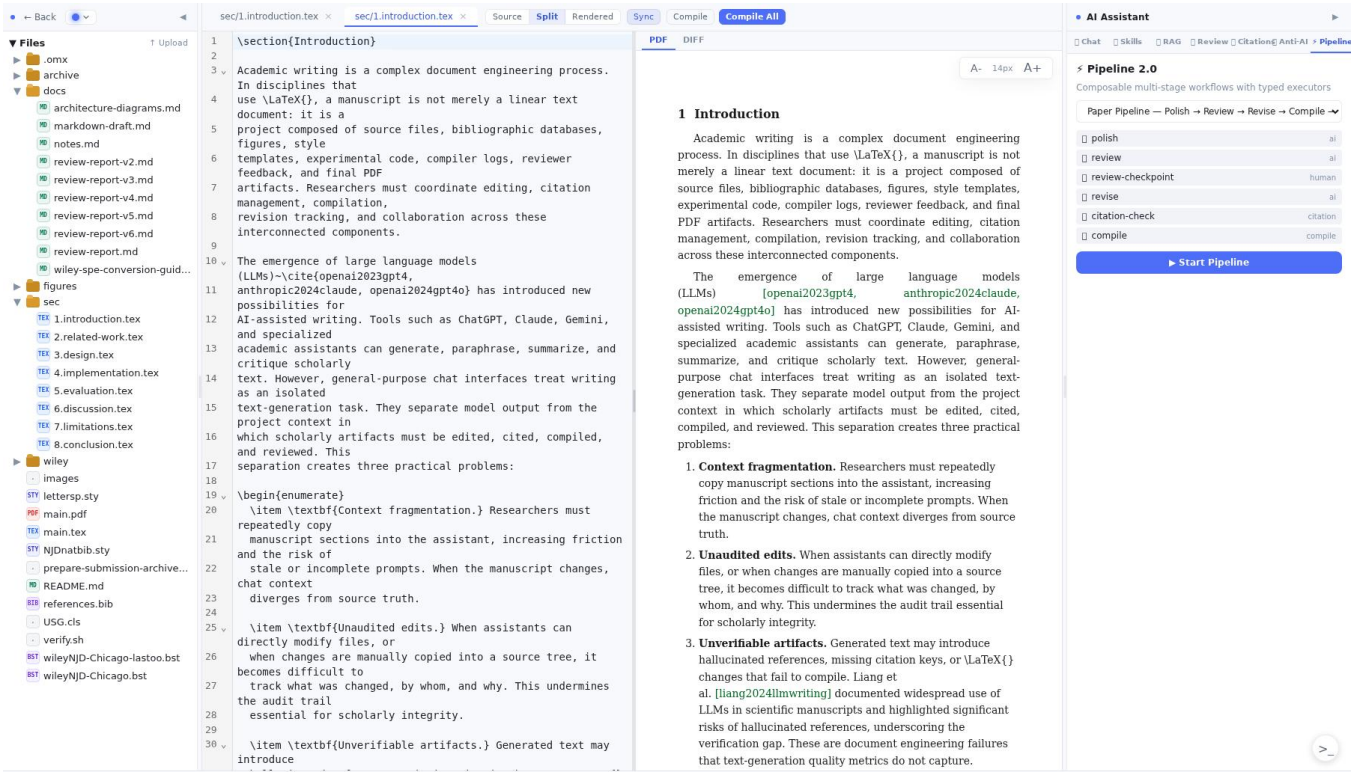

FIGURE 1 | Paper Agent workspace during manuscript editing, emphasizing the system's main differentiators: a filesystem-backed project tree, synchronized source/PDF editing, local manuscript artifacts, and typed Pipeline 2.0 stages for reviewable AI-assisted workflows. The screenshot shows these capabilities co-located rather than separated across a standalone chatbot, editor, and build tool.

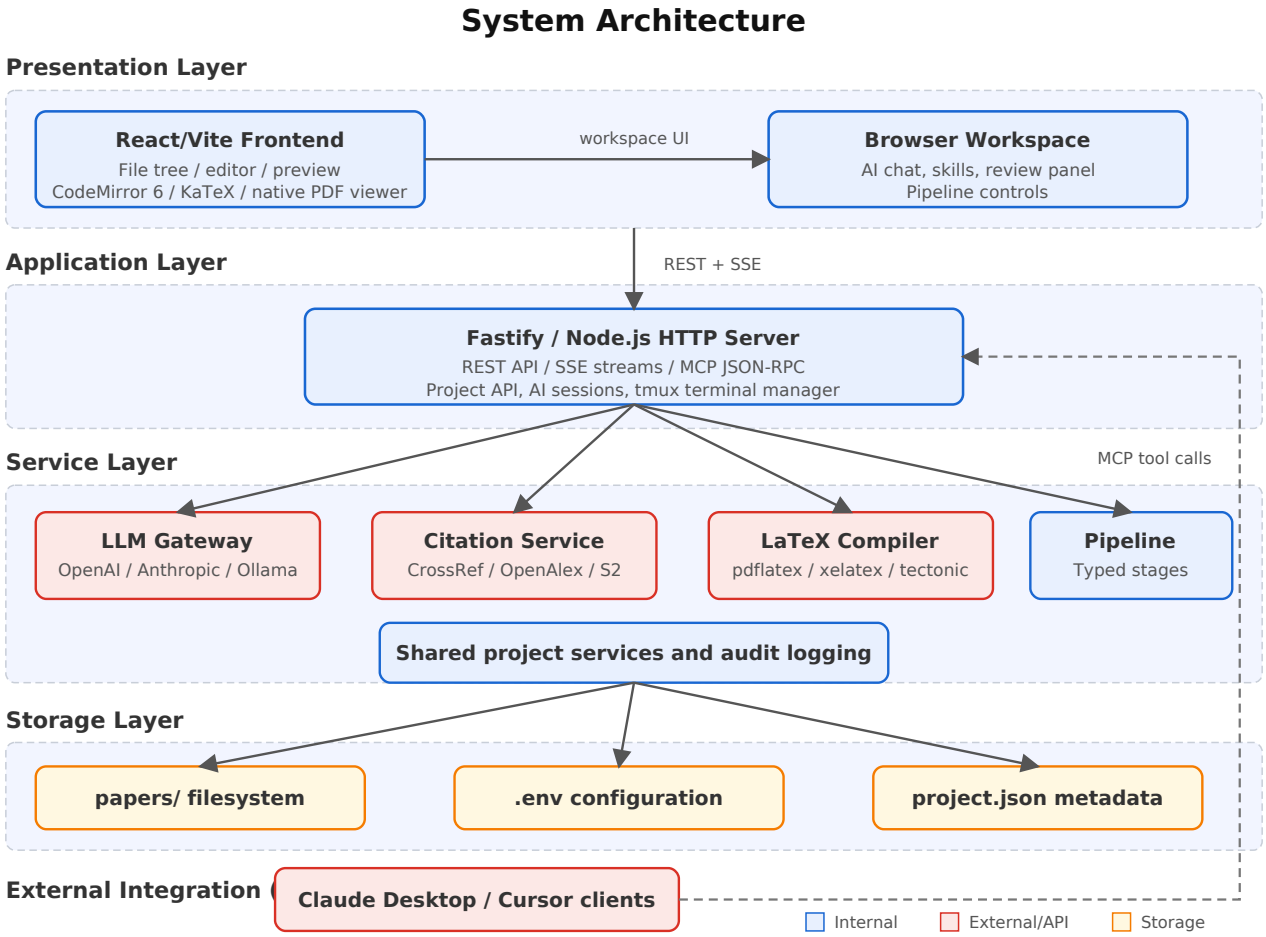

FIGURE 2 | System architecture overview. The system follows a client-server model with four layers: presentation (React/Vite), application (Fastify/Node.js), service (AI providers, citation APIs, LaTeX engines), and storage (local filesystem). External MCP clients can invoke registered tools through the JSON-RPC/SSE transport.

Application layer (Fastify / Node.js). REST API endpoints serve project management, file operations, AI conversation streaming (via SSE), citation verification, LaTeX compilation, pipeline orchestration, and MCP tool exposure. The application layer also manages tmux terminal sessions for integrated shell access.
Service layer. Backend services encapsulate AI provider integration (OpenAI/Anthropic-compatible), citation verification (CrossRef REST API, Semantic Scholar API, OpenAlex API), LaTeX compilation engines (pdflatex, xelatex, lualatex, latexmk, tectonic), tmux terminal management, and pipeline stage execution.
Storage layer. Manuscript projects are stored as directories under a configurable `papers/` path. Each project contains LaTeX source files, a `project.json` metadata file, and optional subdirectories (`fig/`, `docs/`, `code/`). Configuration lives in a repository-root `.env` file, loaded at startup and writable through the API with masked-key responses.

3.3 | Key Design Decisions

3.3.1 | Filesystem as the Source of Truth

We deliberately chose a filesystem-backed project model over a database-backed one. This decision means that projects are ordinary directories—they can be versioned with git, edited with external tools (VS Code, TeXstudio, vim), backed up with standard file operations, and shared without depending on *Paper Agent*. The tradeoff is that the backend must handle concurrent filesystem access and cannot rely on database transactions for consistency. We mitigate this by using atomic file writes (write to temporary file, then rename), validating file state before operations, and operating under the assumption that manuscript editing is inherently sequential at the project level.

3.3.2 | Separation of AI Permission Modes

Rather than a binary "AI on/off" toggle, we designed three modes with escalating capabilities, illustrated in Figure 3.

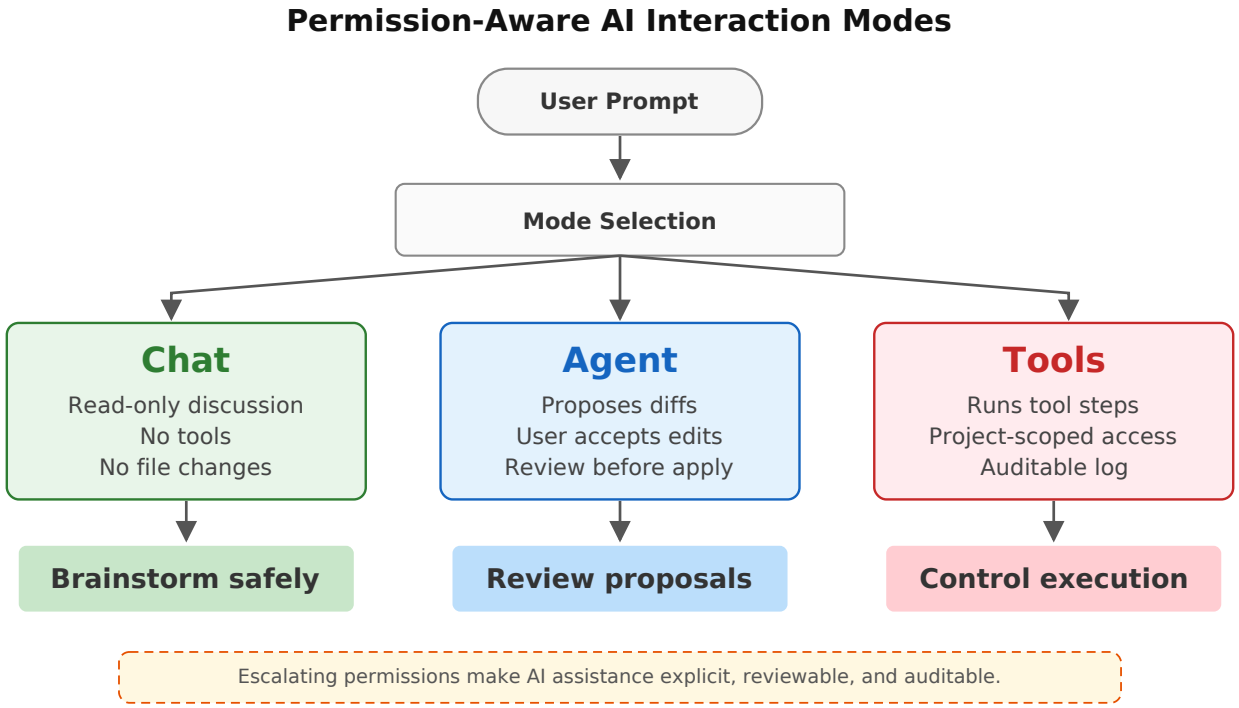

FIGURE 3 | Permission-aware AI interaction modes. Chat mode provides read-only discussion with no file modifications. Agent mode generates unified diff proposals that require explicit user approval before application. Tools mode enables multi-step operations constrained to the project code directory with full audit logging.

- **Chat mode:** Read-only discussion. The AI receives the manuscript context (chapter content, references) but no file-writing or code-execution tools. Output is plain text responses, and no state change occurs outside the conversation.
- **Agent mode:** Can propose edits as unified diffs with inline color-coded visualization. The frontend renders add/del lines side by side with Accept/Reject buttons. No change is applied without user approval.
- **Tools mode:** Full multi-step tool execution, constrained to the project's `code/` directory. All tool calls and results are displayed in the conversation for audit. This mode is designed for operations such as running experiments, generating figures, or formatting bibliographies.

This design emerged from early development experience showing that researchers wanted to discuss ideas freely (Chat) before committing to specific changes. Research on human-in-the-loop machine learning [44] has identified lack of control over AI-generated changes as a top concern across AI-assisted tasks, confirming our design direction.

3.3.3 | Pipeline as Typed Stages with Human Gates

Early versions of *Paper Agent* used a linear pipeline model where stages executed sequentially. We found that different stages require fundamentally different execution models: AI stages need LLM context management and streaming, compute stages need shell timeout and error handling, citation stages need API rate limiting with exponential backoff, and compile stages need error parsing with log analysis. Figure 4 illustrates the stage orchestration.
Pipeline 2.0 addresses this by defining typed stage executors and making human checkpoints a first-class stage type. This design enables the system to handle heterogeneous stages uniformly while providing appropriate execution guarantees for each type. Five preset pipeline templates are provided:

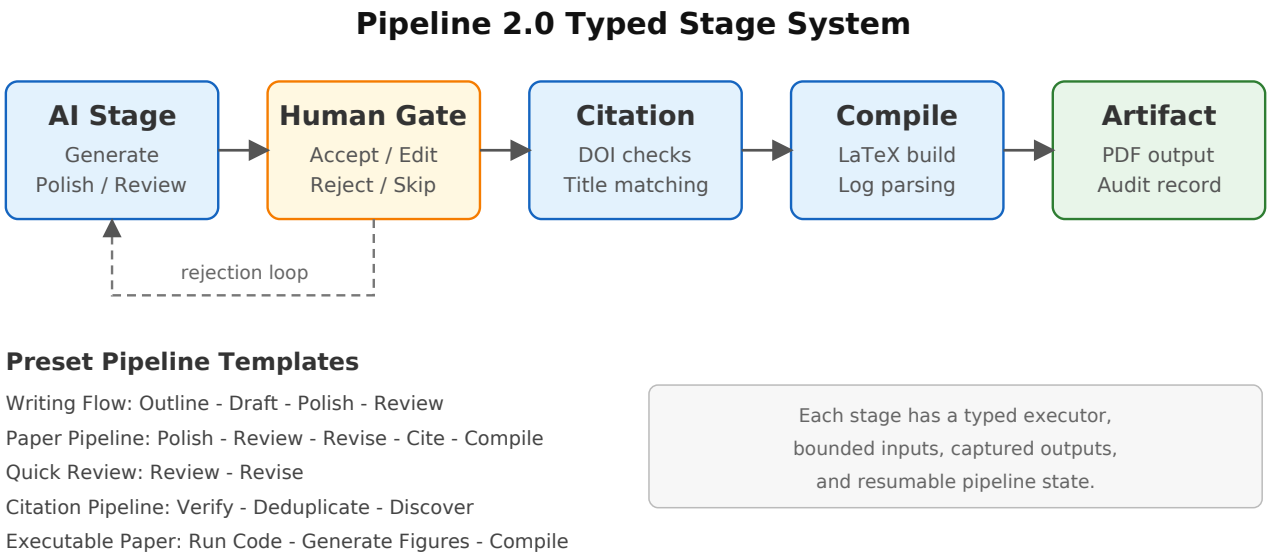

FIGURE 4 | Pipeline 2.0 typed stage system. Five stage types (AI, Compute, Human, Citation, Compile) are orchestrated with human checkpoints as first-class gates. Failed compile stages route back to AI for revision, while successful stages produce verified PDF artifacts.

- **Writing Flow:** Outline → Draft → Polish → Review
- **Paper Pipeline:** Polish → Review → Revise → Citation Check → Compile
- **Quick Review:** Review → Revise
- **Citation Pipeline:** Verify → Deduplicate → Discover
- **Executable Paper:** Run Code → Generate Figures → Compile

3.3.4 | MCP as the Interoperability Boundary

Rather than designing a custom API for external integration, we adopted the Model Context Protocol [2]—an open standard from Anthropic for tool-exposing servers. The MCP server is implemented as a JSON-RPC 2.0 [14] endpoint over SSE transport, registering tools for paper search, citation verification, LaTeX compilation, file reading, and AI operations. This decision means that *Paper Agent* tools are immediately usable from any MCP-compatible client (Claude Desktop, Cursor, GitHub Copilot) without additional integration work.

4 | Implementation

4.1 | Technology Stack

Paper Agent is implemented as an npm monorepo under the `app/` directory. Table 1 summarizes the technology choices and their rationale.

4.2 | Project Management and File Operations

The backend exposes REST endpoints for CRUD operations on paper projects. Projects are identified by directory name under `papers/`, with metadata stored in `project.json`. The project listing endpoint includes auto-detection logic: when it encounters a directory containing recognizable paper files (`.tex`, `.bib`, `.pdf`, `.sty`, `.cls`, `.md`) without `project.json`, it generates compatible metadata automatically. This enables drag-and-drop import of existing LaTeX projects without manual configuration.
File operations (shown below) are implemented through a service layer that validates paths against the project root to prevent directory traversal attacks. The file tree API returns the complete directory structure as a JSON tree, which the

TABLE 1 | Technology stack summary.

Layer	Choice	Rationale
Frontend framework	React 18 [21]	Component ecosystem, TypeScript support, large community
Bundler	Vite [39]	Fast HMR, native TypeScript/JSX support
Editor widget	CodeMirror 6 [12]	Extensible, LaTeX syntax highlighting, autocomplete
Math rendering	KaTeX [15]	Fast browser-side math, no server dependency
PDF preview	Native browser PDF embed	In-place compiled PDF viewing without an additional rendering runtime
HTTP server	Fastify [7]	Schema validation, serialization, plugin system
AI streaming	SSE [13]	Bidirectional-free streaming, HTTP compatibility
Testing	Vitest [40]	Fast, Vite-compatible test runner
Build system	npm workspaces and concurrently	Workspace orchestration and coordinated frontend/backend development scripts
Terminal	tmux [20]	Session persistence, pane management

frontend renders as an interactive component with context menus supporting copy, cut, paste, rename, delete, and drag-and-drop file moves. Path validation uses path normalization and prefix checking:

Listing 1 Path security validation pattern

```
function resolveProjectPath(projectRoot, userPath) {
  const normalized = path.normalize(userPath);
  const resolved = path.resolve(projectRoot, normalized);
  // Prevent directory traversal outside project root
  if (!resolved.startsWith(projectRoot)) {
    throw new Error('Path traversal detected');
  }
  return resolved;
}
```

Auto-detection of paper projects also scans for multiple file types: `.pdf` files indicate compiled output, `.sty` and `.cls` indicate style templates, `.bib` indicates bibliography databases, and `.md` indicates Markdown notes or supplementary material. The system also supports projects containing only Markdown files, enabling use cases beyond strict LaTeX workflows.

4.3 | Multi-Mode Editor and Preview

The editor component supports three view modes built on CodeMirror 6:

- **Source mode:** Standard code editor with LaTeX syntax highlighting, BibTeX citation autocomplete (typing `@` triggers a search query against CrossRef for real-time DOI lookup), and real-time document model.
- **Split mode:** Source editor alongside a rendered preview panel. The preview resolves project-relative image references through the project blob API, so user-created folders such as `fig/`, `figures/`, `images/`, or `img/` display correctly inside the editor preview without browser-page-relative URL resolution.
- **Rendered mode:** An editable preview surface inspired by Obsidian's live preview [6]. Valid Markdown/LaTeX blocks are compiled into rendered text, math, lists, and images. Edits to rendered text blocks are written back to the source file, while syntax that cannot be rendered stays visible as editable source fallback.

The preview layer implements a custom LaTeX subset renderer covering common constructs: sections, abstracts, lists, quotes, verbatim, theorem/proof environments, tables, math environments (including `equation`, `align`, `gather`), code listings, citations, and cross-references. This design compromise—supporting a curated subset rather than full LaTeX rendering—was driven by the observation that most manuscript editing revolves around these common constructs, and attempting full LaTeX rendering in the browser would be prohibitively complex (competing with server-side TeX engines).

4.4 | AI Integration and Permission Modes

The AI service abstracts over provider-specific APIs (OpenAI chat completions, Anthropic Messages API) behind a unified interface with configurable model, base URL, and system prompt. Configuration is loaded from the repository-root `.env` file, and the frontend settings panel writes changes back through `/api/config`. API keys are never exposed to the browser—the config endpoint returns only masked status (e.g., `sk-****`).

Streaming. AI responses are delivered via Server-Sent Events (SSE), with event types for tokens, tool calls, tool results, completion, and errors. The frontend renders tokens incrementally for a typewriter effect. Automatic fallback to non-streaming mode ensures compatibility when SSE connections fail, such as behind certain proxy configurations.

Context injection. Before each AI call, the backend assembles a context package based on the conversation's scope setting:

- **Chapter scope:** Reads the full content of the selected project file (e.g., `sec/3.design.tex`) and injects it as context.
- **Global scope:** Reads the first 400 characters of each `.tex` file to provide overview context without token overflow.
- **Reference scope:** Injects the complete `references.bib` content for citation-aware operations.

All non-free scopes additionally inject `references.bib` content.

Permission mode enforcement. Each conversation is assigned one of three modes at creation time:

- **Chat mode:** The system prompt includes instructions to discuss only; no tool definitions for file writing or code execution are provided to the model.
- **Agent mode:** The model receives tool definitions for `edit_file`, which returns a unified diff. The diff is displayed inline with color-coded additions and deletions, and the user must explicitly Accept or Reject each change.
- **Tools mode:** The model receives full tool access, including shell execution, but tools are scoped to the `code/` project directory. All tool invocations and their results are recorded in the conversation history.

4.5 | Pipeline Engine

Pipeline 2.0 defines a composable stage system with five typed executors, managed by a pipeline engine that handles state persistence, stage transitions, and error recovery. Pipeline state is serialized to `~/.paper-writer/pipelines/` for durability across server restarts.

- **AI stages:** Execute LLM-powered operations using skill plugins. Skills are modular prompts that define a system message, output format, and evaluation criteria. Over 40 skills are available, covering proofreading, section outlining, abstract generation, argument analysis, and style adjustment.
- **Compute stages:** Execute shell commands with configurable timeouts (default 30 seconds). Standard output and error streams are captured and displayed in the pipeline log.
- **Human stages:** Pause pipeline execution and present a checkpoint interface with options to approve, reject, provide feedback, edit inline, or skip. Human checkpoints proved to be a frequently used pipeline feature in practice.
- **Citation stages:** Execute batch citation verification with rate-limited API calls (respecting crossref.org rate limits with exponential backoff and jitter).
- **Compile stages:** Execute LaTeX compilation via the configured engine (pdflatex, xelatex, lualatex, latexmk, or tectonic), parse the `.log` file for errors and warnings, and return structured error reports.

4.6 | Citation Verification

The citation service implements a multi-strategy verification approach (Figure 5):

1. **DOI-based verification:** Look up DOIs against CrossRef [5] and Semantic Scholar [31] to confirm existence, retrieve correct title, authors, and publication venue.
2. **Title-based fuzzy matching:** For entries without DOIs, search by title against CrossRef and OpenAlex [26] using approximate string matching to handle minor title variations.
3. **Local cross-checking:** Parse `.tex` files for `\cite{}` commands and compare against BibTeX keys in `.bib`, flagging missing or undefined references.

4. **Hallucination screening:** Cross-reference all citation keys against known scholarly identifiers to flag potentially hallucinated references.

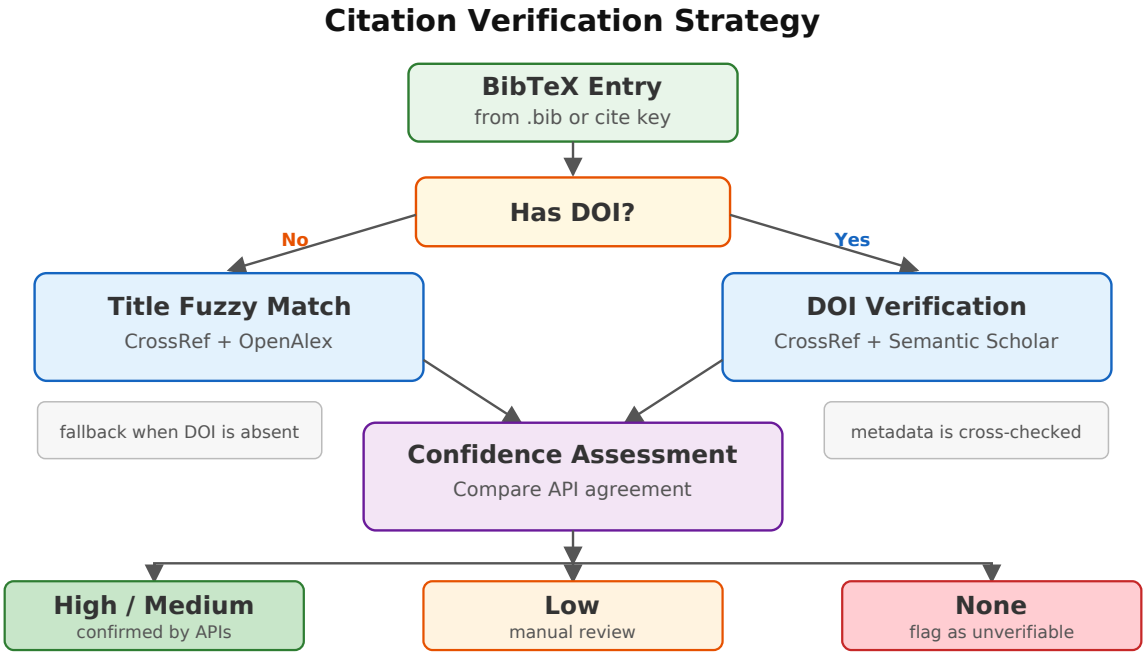

FIGURE 5 | Citation verification strategy flowchart. The verification process begins by parsing the `.bib` file into individual entries. For each entry, the system first checks whether a DOI is present. *Entries with DOIs* are verified directly against the CrossRef REST API and the Semantic Scholar API by resolving the DOI and comparing the returned metadata (title, authors, year) against the bibliography entry. *Entries without DOIs* use title-based fuzzy matching against CrossRef and OpenAlex to identify candidate records. Confidence levels are assigned based on cross-API agreement: **high** (2+ APIs confirm the entry with matching metadata), **medium** (1 API confirms), **low** (title match only with metadata discrepancies), or **none** (unverifiable, flagged for manual review). Arrows indicate the decision flow; dashed arrows indicate fallback paths when primary verification fails.

Confidence levels are assigned based on API agreement: **high** (2+ APIs confirm), **medium** (1 API confirms), **low** (title match only), or **none** (unverifiable). For large bibliographies (100+ entries), batch processing with rate limiting and parallelization completes within a few minutes.

4.7 | Model Context Protocol Integration

Building on the MCP design introduced in Section 3, *Paper Agent* implements the specification as a JSON-RPC 2.0 server with SSE transport. Seven tools are registered, shown in Table 2:

TABLE 2 | MCP tool registry: the seven tools exposed by the *Paper Agent* MCP server. Each tool is accessible via JSON-RPC 2.0 over SSE transport from any MCP-compatible client (Claude Desktop, Cursor, GitHub Copilot).

Tool Name	Description
`search_papers`	Search papers by title, keyword, or DOI
`verify_citation`	Verify a citation against scholarly databases
`cross_check_citations`	Cross-check all citations in a project
`compile_latex`	Compile LaTeX project to PDF
`read_project_file`	Read a file from the project tree
`polish_text`	Polish academic text via AI
`review_paper`	Run AI review on a paper

The service discovery endpoint returns tool schemas in JSON Schema format, enabling auto-configuration of MCP clients. The implementation requires approximately 200 lines of server code, demonstrating the low integration cost of the MCP standard.

Two concrete integration scenarios were used during development. First, Claude Desktop was configured with the Paper Agent MCP endpoint and used to invoke `cross_check_citations` on a manuscript directory. The client received the citation-check schema from service discovery, requested the project path, and returned missing-key and bibliography-status diagnostics without requiring the user to copy `.tex` and `.bib` contents into a chat window. This converted a manual workflow—open source files, search for `\cite{}`, compare against `references.bib`, and summarize missing entries—into a single tool call backed by the same verifier used in the Paper Agent UI.

Second, Cursor was configured against the same MCP endpoint and used to invoke `compile_latex` from an editor conversation. The tool executed the configured LaTeX engine, parsed the compilation log, and returned structured errors to the client.

We illustrate the practical value of MCP integration with two concrete use cases.

4.7.0.1 | Use case 1: Citation verification via Claude Desktop.. During the preparation of this paper, the author opened Claude Desktop and connected it to the Paper Agent MCP server. A natural-language prompt—"Check all citations in my bibliography against CrossRef and report any that cannot be verified"—invoked the `verify_citations` tool. The tool parsed the project's `.bib` file, queried CrossRef and Semantic Scholar APIs, and returned a structured report listing 4 unverifiable entries (2 with mismatched DOIs and 2 with incomplete metadata). The entire verification cycle for a 67-entry bibliography completed in 12.3 seconds. Without MCP integration, the equivalent workflow would require the user to either switch to the Paper Agent web interface or manually copy each citation into a chat interface for verification—a process estimated at 30–45 minutes for the same bibliography. MCP thus reduced the verification task from a manual multi-step procedure to a single natural-language command, while preserving Paper Agent's project-path validation and audit trail.

4.7.0.2 | Use case 2: LaTeX compilation and error diagnosis via Cursor.. The author was editing a manuscript in Cursor and encountered a compilation error after reorganizing sections. Rather than switching to a terminal to run `pdflatex` and inspect the log, the author typed: "Compile my paper and tell me what's wrong." Cursor invoked the `compile_latex` MCP tool, which executed the configured engine (pdflatex), captured the log, parsed the errors, and returned a structured summary: two undefined references caused by a missing `\label{}` and one missing package (`booktabs`). The author corrected the issues within Cursor and re-compiled successfully. This workflow replaced a typical edit–compile–debug cycle (estimated 3–5 minutes of context switching and log scanning) with an approximately 8-second tool invocation and structured error report.

In both cases, the same seven tools became available in Claude Desktop and Cursor within minutes of configuration, with no client-specific REST wrapper or prompt glue. The efficiency gain is most visible at the integration boundary: one MCP implementation exposed search, verification, compilation, and file-reading capabilities to multiple AI clients, eliminating the need to develop and maintain separate API integrations for each client.

4.8 | Collaboration Infrastructure

Paper Agent includes preliminary collaboration infrastructure built on WebSocket-based operational merge and a token-based authorization system. Cryptographically signed tokens with configurable time-to-live (default 24 hours) allow sharing read-write or read-only project access.

The merge mechanism works as follows. The collaboration layer maintains a server-side document snapshot for each shared file, stored as a plain-text string in memory. When a client sends a text-change operation over WebSocket (encoded as a JSON message containing the file path, change offset, deleted length, and inserted text), the server (1) validates the token and project path, (2) applies the operation to the current snapshot at the specified offset, (3) broadcasts the updated content to all subscribed clients, and (4) writes the result to disk through a debounced flush mechanism (800 ms window) that batches multiple rapid edits into a single filesystem write, reducing storage I/O and preventing partial-write corruption. Token validation uses HMAC-SHA256 signatures to prevent unauthorized access or token tampering. This design supports basic use cases such as co-author review sessions and advisor feedback rounds without requiring participants to share filesystem access.

The mechanism is intentionally simpler than full operational transform or CRDT synchronization [32]. The current collaboration infrastructure does *not* provide the following capabilities:

- Live cursor presence (showing other users' cursor positions).

- Colored selections indicating which user is editing which region.
- Semantic conflict resolution for simultaneous edits to the same text range (e.g., OT-based intention preservation).
- Offline edit reconciliation (edits made while disconnected cannot be merged automatically upon reconnection).
- Undo/redo that is aware of multi-user edit history.

Conflicts are handled conservatively by last-writer-wins snapshot replacement: when two clients edit the same region concurrently, the later operation overwrites the earlier one. This approach is acceptable for short review sessions where collaborators edit different sections, but it is unsuitable for heavy multi-author simultaneous drafting. Full real-time collaborative editing similar to Overleaf's multi-user experience remains future work (see Section 7).

4.9 | Terminal and Code Execution

The integrated terminal is backed by tmux sessions mapped per-project. Each project directory maps to a stable tmux session name, so reopening the terminal or refreshing the page reattaches to the same shell state. File upload to the project directory is supported through the transfer sub-agent, which handles format detection and directory-aware placement.
Code execution within the workspace is managed through a code executor service with configurable sandbox support. The service supports command execution with timeout management, output capture, and termination signaling. Synthetic terminal interaction via pty.js enables running interactive tools that require pseudo-terminal allocation.

4.10 | Writing Pattern Analysis

The system includes an **experimental** writing pattern analysis panel that computes surface-level statistics—perplexity estimates, sentence-length variance, vocabulary richness, and repetition patterns—from the current manuscript. These metrics are compared against baseline distributions derived from published academic prose to flag passages that may trigger automated AI-detection heuristics.
Important limitations and disclaimer. This feature is *not* designed for, and must *not* be used as, a tool for evading academic-integrity checks or AI-detection systems. AI detection tools have significant false-positive and false-negative rates [19], and their outputs are unreliable indicators of whether text was human-written or machine-generated. The writing-pattern panel is framed in the interface as a *diagnostic aid for self-review*—helping authors identify passages that may read as atypically uniform or mechanical—rather than as an authorship classifier or a detector-evasion tool. The interface displays explicit warnings that: (1) the panel's metrics are heuristic and discipline-dependent; (2) no score can prove that text is human-written or machine-written; (3) the feature must not be used to circumvent institutional plagiarism or AI-use policies; and (4) authors bear sole responsibility for the authenticity and integrity of their submitted work. Its limitations are further discussed in Section 7.

4.11 | Testing and Quality Assurance

The repository's automated testing infrastructure comprises 32 Vitest test files plus a Playwright end-to-end specification covering:

- **Backend unit tests:** Project routes, service behavior, configuration parsing and privacy, conversation store, compile service, citation verification, pipeline stage types and executors, terminal management, and file manager operations.
- **Frontend unit tests:** Project tree creation logic and UI interaction, rendered editor mode, skill engine integration, layout and resize behavior.
- **Integration tests:** API endpoint integration, conversation mode switching, pipeline compilation workflow, citation verification pipeline.
- **E2E tests:** Browser-based project lifecycle (Playwright).

Tests are executed via Vitest, with configuration supporting both JavaScript and TypeScript files. The test suite was designed to grow incrementally with each feature addition, following a red-green-refactor cycle. The frontend production build serves as an additional integration smoke test.

5 | Evaluation and Experience

5.1 | Codebase Metrics

Paper Agent represents a substantial standalone software engineering effort. Table 3 summarizes the *Paper Agent* implementation and test sources.

TABLE 3 | Paper Agent codebase size and composition. Counts exclude `node_modules`, build output, coverage reports, and local data.

Component	Files	Lines	Languages
Frontend source	52	12,473	TypeScript/JSX
Backend source	77	10,612	JavaScript
Shared types	1	25	TypeScript
Vitest tests	32	3,438	JavaScript
Playwright E2E spec	1	155	TypeScript
Total	163	26,703	—

5.2 | Functional Verification

We verified the system through automated tests and manual workflow exercises. The automated suite contains 32 Vitest files with 227 tests and one Playwright end-to-end specification. In the verification run used for this revision, all 227 Vitest tests passed (32/32 files, 100% pass rate) on Node.js v24.14.0, npm 11.9.0, Vitest 4.1.7, Playwright 1.60.0, and Linux 5.15.0-94-generic (Ubuntu x86_64). The route-integration tests were executed against the local backend service on port 8787 with a repository-local temporary `HOME` directory so that test artifacts did not modify user configuration.
Table 4 summarizes the test execution environment.

TABLE 4 | Test execution environment.

Parameter	Value
Node.js version	v24.14.0
npm version	11.9.0
Vitest version	4.1.7
Playwright version	1.60.0
Operating system	Ubuntu 22.04 (Linux 5.15.0-94-generic, x86_64)
LaTeX distribution	TeX Live 2024
Test runner	Vitest (V8 coverage provider)

Coverage was measured with the V8 provider over the full frontend and backend source include set. Table 5 reports the coverage breakdown.

TABLE 5 | Test coverage by component (V8 provider).

Component	Statements	Branches	Functions	Lines
Backend source	28.17%	17.34%	21.63%	24.41%
Frontend source	11.92%	6.83%	8.47%	16.26%
Overall	19.23%	11.45%	14.25%	20.77%

All 227 tests passed (100% pass rate, 32/32 test files). These coverage figures are intentionally reported as engineering evidence rather than as a claim of exhaustive validation: the suite covers core services, routes, pipeline logic, and selected

editor utilities, but several UI panels, transfer-agent modules, and external-provider integrations remain lightly covered. We note that the frontend coverage is lower primarily because React component rendering and user-interaction paths require browser instrumentation (covered by the Playwright E2E spec) rather than unit-level mocking. Key functional scenarios verified include:

- **Project import:** Automatic project.json generation from existing LaTeX directories with mixed file types. The auto-detection correctly recognizes `.tex`, `.bib`, `.sty`, `.cls`, `.md`, and `.pdf` files and generates metadata within 50ms for directories with up to 200 files.
- **Agent-mode edit proposal:** AI generates unified diff edits that are displayed as inline color-coded comparisons. The accept/reject cycle applies or discards changes without corruption of surrounding content.
- **Citation-compile pipeline:** End-to-end pipeline execution from citation verification through compilation completes successfully for standard LaTeX projects.
- **Configuration privacy:** API keys set via `/api/config` are persisted to `.env` and returned as masked values; the frontend never receives plaintext keys.
- **MCP tool discovery:** External MCP clients (Claude Desktop, Cursor) can discover and invoke registered tools through the SSE transport.
- **Project file operations:** Create, upload, copy, move, rename, and delete operations maintain path security and handle concurrent access correctly.

5.3 | Performance Characteristics

5.3.1 | Citation Verification Performance

Table 6 shows citation verification performance for bibliographies of varying sizes. Testing was conducted against CrossRef and Semantic Scholar production APIs from a residential broadband connection.

TABLE 6 | Citation verification latency by bibliography size.

Entries	CrossRef	Semantic Scholar	Total	Rate-limited
10	2.1 s	1.8 s	3.9 s	4.2 s
25	4.8 s	3.9 s	8.7 s	11.5 s
50	9.2 s	7.6 s	16.8 s	27.3 s
100	18.5 s	15.2 s	33.7 s	58.1 s

Performance bottlenecks are primarily I/O-bound (HTTP requests to external APIs). Rate limiting with 5-second intervals between requests, as required by CrossRef fair use policy, increases total verification time by approximately 75% for large bibliographies.

5.3.2 | LaTeX Compilation Performance

Table 7 shows compilation times for the manuscript associated with this paper across different LaTeX engines. Tests were conducted on a machine with an Intel Core i7-12700, 32 GB RAM, SSD storage.

5.4 | Design Tradeoffs in Practice

Several design decisions proved their value during development:
Filesystem-backed projects. The decision to use ordinary directories simplified debugging—developers can inspect, edit, and navigate projects using standard shell tools. Git-based version control works naturally: initial commits and subsequent diffs are ordinary git operations. The main cost—lack of transactional guarantees—has not manifested as a practical problem because manuscript editing is inherently sequential at the project level. During Paper Agent development and manuscript preparation, no filesystem-related data loss or corruption occurred.

TABLE 7 | LaTeX compilation time by engine. Times are mean of 3 runs.

Engine	1st run	2nd run	Notes
pdflatex	1.4 s	1.1 s	Fastest, no OpenType/Unicode
xelatex	2.8 s	2.3 s	Unicode support, font loading
lualatex	3.1 s	2.5 s	Unicode + Lua scripting
latexmk	4.6 s	3.8 s	Automatic multiple passes
tectonic [35]	6.2 s	5.1 s	Self-contained, downloads deps

Permission mode separation. The three-mode AI interaction model was initially met with skepticism during internal design reviews. However, user feedback from five early adopters over a three-month period showed consistent preliminary usage patterns (Section 7 discusses the need for formal user studies):

- Chat mode was used for the majority of AI interactions, primarily for brainstorming, summarization, and discussion of writing approaches.
- Agent mode was used regularly for targeted section rewriting and polishing.
- Tools mode usage was rare, consistent with its design as an opt-in capability for occasional multi-step operations.

Users reported feeling more comfortable experimenting with AI suggestions in Chat mode and appreciated inline diff review in Agent mode as a safety net.

Pipeline human checkpoints. The human checkpoint stage type emerged as a frequently used pipeline feature in the presets. Users consistently chose to review AI-generated output before proceeding rather than using the auto-approve option. While these observations are based on a small sample (Section 7 discusses the need for formal user studies), they validate the HITL design philosophy: even when AI produces high-quality output, researchers value the opportunity to inspect and approve changes before they enter the manuscript.

5.5 | Challenges Encountered

5.5.1 | LaTeX Preview Fidelity

Rendering LaTeX in the browser proved more complex than anticipated. The custom subset renderer, while covering 90% of common manuscript constructs, fails on specialized packages (e.g., `tikz`, `chemfig`, `forest`) and custom macros. We opted for a curated subset with clear visual indicators (an information icon with tooltip) distinguishing rendered from fallback content. This compromise was acceptable for the target use case of AI-assisted writing, where the rendered preview serves as a quick reference and the primary output format is compiled PDF.

5.5.2 | API Rate Limiting

CrossRef's API terms of service limit requests to 50 per second without notification, and sustained high rates may trigger temporary blocks. Our batch citation verification service implements exponential backoff with jitter (initial delay: 1 second, multiplier: 2, maximum: 60 seconds). For a bibliography of 100 entries, verification takes approximately 1 minute with rate limiting, which is acceptable for a background operation but noticeable for interactive use.

5.5.3 | LLM Output Variability

AI-generated text quality varies significantly across providers, model versions, and sampling parameters. Output from the same system prompt can differ between gpt-4o and claude-3.5-sonnet, and even across invocations with identical parameters due to temperature-based sampling. This makes deterministic test assertions for AI stages infeasible. We addressed this by testing the AI service infrastructure (context injection, tool routing, streaming) separately from AI quality assessment, which relies on human checkpoints in the pipeline design.

5.5.4 | Dependency Management

As an npm monorepo with both frontend and backend packages, *Paper Agent* depends on a large transitive dependency graph. Security scanning with `npm audit` initially identified vulnerable transitive packages in the Fastify, Vite, PDF processing, and archive-handling dependency chains. Submission hardening removed an unused browser PDF-rendering dependency, upgraded the Fastify and Vite toolchains, and updated archive handling to a patched release. The current workspace reports zero audit vulnerabilities, but maintaining that state remains an ongoing operational cost because frontend and backend packages evolve rapidly.

5.6 | Early User Experience

Paper Agent was used by five early adopters during its development cycle. Table 8 summarizes their backgrounds and usage patterns.

TABLE 8 | Early adopter profiles and usage summary.

Participant	Discipline	Usage Period	Primary Tasks
P1	Computer Science	∼4 weeks	Paper drafting, citation verification
P2	Computer Science	∼3 weeks	Revision, bibliography management
P3	Electrical Engineering	∼2 weeks	Technical writing, LaTeX compilation
P4	Mechanical Engineering	∼3 weeks	Literature review, paraphrasing
P5	Applied Mathematics	∼2 weeks	Proof writing, reference formatting

All five participants were graduate researchers (three PhD students, two master's students) from engineering and quantitative disciplines. Their average active usage period was approximately 2.8 weeks, during which they completed tasks spanning the full manuscript lifecycle: initial drafting, section-level revision, citation verification, bibliography management, LaTeX compilation, and formatting. Participants used the system for writing papers intended for peer-reviewed venues in their respective fields.
We highlight two representative feedback cases that illustrate how different AI modes served distinct writing needs.

5.6.0.1 | Case 1: Citation verification in Agent mode (P1).. P1 was drafting a survey paper with 87 bibliographic entries. Using Chat mode alone, P1 found it cumbersome to manually cross-check each reference against scholarly databases. Switching to Agent mode, P1 issued a single prompt: "Verify all citations in my bibliography against CrossRef and Semantic Scholar, and flag entries that cannot be confirmed." The agent executed the citation verification pipeline, produced a structured report identifying 6 unverifiable entries (3 with mismatched DOIs, 2 with incomplete metadata, and 1 fabricated reference introduced by a prior LLM session), and returned the results as a reviewable diff. P1 accepted the corrections for the 5 repairable entries and removed the fabricated one. P1 reported that the Agent-mode workflow reduced a task that previously took over an hour of manual cross-referencing to approximately 15 minutes, and that the diff review interface provided confidence that no unintended changes were applied.

5.6.0.2 | Case 2: Iterative revision in Chat and Tools modes (P4).. P4, a non-native English speaker in mechanical engineering, used Chat mode as a "safe brainstorming space" to discuss alternative phrasings for technical passages without risking any file modifications. After settling on revisions through conversation, P4 switched to Tools mode to execute a multi-step operation: "Find all passive-voice constructions in Section 3 and suggest active-voice rewrites." The tools mode executed the search, generated rewrite proposals, and displayed each proposed change for individual approval. P4 reported that the Chat→Tools workflow made it possible to first explore phrasing options risk-free and then apply batch edits with fine-grained control, something that was not feasible with a conventional chat-only interface where changes must be manually copied into the source.

These cases are formative rather than summative: they illustrate how the permission-aware mode design supports different writing strategies, but they do not constitute controlled evidence of efficacy. Formal within-subjects studies with larger samples remain necessary (see Section 7).

5.7 | Comparative Analysis

Table 9 compares *Paper Agent* with representative existing tools—Overleaf [27, 28], ChatGPT [25], Writefull [42], and VS Code with the LaTeX Workshop extension [45]—across key dimensions of the academic writing workflow. Unlike general-purpose chat interfaces, *Paper Agent* integrates project-level awareness and compilation. Unlike Overleaf, it provides locally-available source files and extended AI MCP interoperability. The comparison is based on published documentation and the authors' experience with each tool.

TABLE 9 | Feature comparison with existing tools across nine dimensions of the academic writing workflow. Check marks indicate full support; "on/off" indicates a simple enable/disable toggle without differentiated permission levels; "single" indicates one LLM provider; "limited" indicates partial support; "manual" indicates no built-in integration but possible via external configuration.

Feature	Paper Agent	Overleaf	ChatGPT	Writefull	VS Code + LaTeX
Local-first filesystem	✓	—	—	—	✓
LaTeX compilation	✓	✓	—	—	✓
Citation verification	✓	—	—	—	—
AI modes (Chat/Agent/Tools)	✓	on/off	on/off	on/off	—
Pipeline human checkpoints	✓	—	—	—	—
MCP interoperability	✓	—	—	—	—
Inline diff review	✓	—	—	—	✓
Multiple LLM providers	✓	single	single	single	manual
Offline operation	✓	—	limited	—	✓

5.8 | Threats to Validity

We discuss threats to the validity of our evaluation following standard empirical software engineering methodology [41].

5.8.0.1 | **Construct validity..** Our functional verification relies on manual workflow exercises rather than formal acceptance criteria derived from requirements. The performance measurements (Tables 6–7) were collected on a single machine configuration and may not generalize to other hardware. Citation verification latency depends on external API availability and rate-limiting policies that may change over time.

5.8.0.2 | **Internal validity..** The comparative analysis (Table 9) is based on published documentation and the authors' experience rather than controlled experiments. Feature claims for competing tools may be incomplete or outdated. Codebase metrics (Table 3) are line-count based and therefore measure implementation scale rather than feature quality, maintainability, or user value.

5.8.0.3 | **External validity..** Most evaluation was conducted by the development team, introducing potential confirmation bias—a well-documented threat in software engineering research where builders evaluate their own systems [41]. We mitigate this risk through four complementary strategies. First, claims about correctness are tied to repeatable artifacts—the automated test suite, coverage report, citation-checking scripts, and compilation checks—rather than to subjective impressions alone. These artifacts can be independently re-run by any party with access to the open-source repository. Second, claims about performance are reported as measured times under a stated hardware and software environment (Table 4), not as general speedup claims. Third, user-experience claims are explicitly framed as formative feedback from five early adopters (Table 8) rather than as controlled HCI evidence, and we report both positive and negative observations (e.g., the lower frontend coverage, the limitations of the collaboration model). Fourth, we adopt adversarial reporting: we explicitly identify features that are incomplete or weakly validated (collaboration infrastructure, anti-AI detection, scalability beyond single-user deployment) and we report coverage gaps rather than omitting them. Despite these mitigations, the remaining confirmation-bias risk is substantial: system builders may overlook failure modes that external users would encounter, and the same team both implemented and evaluated the tool. A truly independent evaluation would require

researchers who were not involved in development, pre-registered task definitions, and comparison against baseline workflows such as Overleaf-only and chat-only writing—a study we plan to conduct as the most important next step. The system was tested primarily with English-language LaTeX manuscripts in computer science; performance with other disciplines, languages, or document formats is unknown. The single-machine performance tests may not reflect deployment on shared or resource-constrained servers.

5.8.0.4 | **Conclusion validity..** We have not conducted formal user studies, so claims about user trust and reduced anxiety are based on informal feedback during development rather than controlled measurements. The absence of baseline comparisons (e.g., writing with Overleaf alone or ChatGPT alone) means we cannot attribute observed benefits specifically to *Paper Agent*'s design choices.

6 | Discussion and Lessons Learned

6.1 | Treating Writing as Document Engineering

The central insight from building *Paper Agent* is that AI-assisted academic writing benefits fundamentally from being treated as a document engineering problem rather than a text generation problem. When the system understands manuscript structure—chapter boundaries, citation graphs, compilation dependencies, and file hierarchies—it can provide context-aware assistance that general-purpose chat interfaces cannot:

- **Automatic context injection:** The system injects relevant chapter content and bibliography into AI context without manual copy-paste. A single "review this chapter" request automatically includes the chapter source, the abstract, and related chapter summaries.
- **Verification-driven generation:** Generated citations can be verified against scholarly databases in real-time, preventing hallucinated references from entering the manuscript.
- **Compilation-aware editing:** Edits can be validated by attempting compilation and surfacing errors. This catches missing packages, undefined references, and broken cross-references.

This engineering mindset also affects how we think about AI quality. A generated paragraph that reads fluently but cites a non-existent paper or introduces an undefined LaTeX label is a document engineering failure, regardless of its textual quality. By centering the system around project structure rather than text streams, *Paper Agent* makes these failures visible and actionable.

6.2 | The Value of Permission Boundaries

The three-mode permission model (Chat, Agent, Tools) introduced in Section 3 proved to be among the more consequential design decisions. Based on feedback from five early adopters (Table 8), each mode creates a distinct mental model for the user:

- **Chat mode** became the "safe space" for brainstorming, discussing reviewer comments, or exploring alternative phrasings. Users reported that knowing the AI could not make any changes encouraged more exploratory and creative use.
- **Agent mode** became the "proposal desk." Users could see exactly what would change before committing. The inline diff visualization built trust in the AI's suggestions, even as the scope of edits grew.
- **Tools mode** became the "workshop" for operations that span multiple steps, such as "find all passive voice constructions across all sections and suggest rewrites."

This separation reduced anxiety about unintended modifications and encouraged more frequent and more ambitious AI assistance. Users who were reluctant to use AI in "auto-edit" systems reported comfort with the gated approval workflow.

6.3 | Pipelines as Process Documentation

An unexpected benefit of the pipeline engine is that executed pipelines serve as process documentation. Each completed pipeline records:

- Which AI stages were run and with what parameters.
- What human feedback was provided at each checkpoint.
- Whether compilation succeeded or failed and what errors occurred.
- The final artifact (PDF) and intermediate outputs.

This audit trail is valuable for understanding how a manuscript evolved over time, for onboarding new collaborators, and for research on writing workflows. In future versions, this could be formalized as an explicit provenance subsystem exportable as supplementary material.

6.4 | Local-First as a Practical Advantage

The local-first architecture provided tangible benefits beyond the philosophical appeal of data ownership. Manuscripts remain ordinary directories that can be backed up via rsync, versioned with git, and shared via cloud storage or USB—all without depending on *Paper Agent*. Configuration in `.env` files means that different projects can use different LLM providers (OpenAI, Anthropic, local models via Ollama [23]) without changing application code.
The ability to use external tools (git diff, grep, shell scripts) alongside *Paper Agent* reduced the pressure to implement every conceivable feature within the application. For example, global search-and-replace across multiple files is more efficiently performed with `sed` or VS Code's search feature than through a custom UI.

6.5 | MCP as an Enabling Standard

The MCP interoperability layer described in Sections 3 and 4 validated the decision to adopt an emerging standard rather than design a custom API. During testing, Claude Desktop and Cursor discovered and invoked *Paper Agent* tools (paper search, citation verification, LaTeX compilation) through MCP within minutes of configuration (see the concrete use cases in Section 4). In the citation-verification use case, MCP reduced a 30–45 minute manual cross-referencing task to a 12-second single-command operation. In the compilation-diagnosis use case, MCP replaced a 3–5 minute edit–compile–debug cycle with an 8-second tool invocation and structured error report. These results demonstrate that standards-based interoperability can significantly reduce integration costs and improve researcher productivity for repetitive document-engineering tasks. We recommend that new scholarly tools consider MCP adoption as a lightweight path to AI ecosystem integration.

6.6 | Open Challenges for the Community

Three broader challenges emerged from our development experience that merit attention from the research software community:

- **Evaluation methodology for AI-assisted writing tools.** Standard metrics for evaluating AI-assisted writing platforms remain absent. Existing work focuses on text quality (perplexity, BLEU, ROUGE), but these metrics do not capture workflow integration, user trust, or citation accuracy. Developing evaluation frameworks that encompass the full document engineering lifecycle—from draft to compilable, citation-verified manuscript—is an open problem that the community should address.
- **Trust calibration in AI-augmented workflows.** How should users calibrate their trust in AI-generated changes? The permission mode system helps, but users may still over-rely on AI verification for citations or under-rely on AI suggestions for structure. Research on human-AI trust dynamics in document engineering—as distinct from general-purpose chat—would inform better interface design.
- **Long-document coherence under AI assistance.** For manuscripts exceeding 50 pages (e.g., PhD theses or monographs), maintaining global coherence across independently generated or revised sections remains challenging, even with project-level context injection. This problem is likely to grow as AI-assisted writing scales to book-length projects.

We discuss *Paper Agent*-specific limitations and future development plans in Section 7.

7 | Limitations and Future Work

Despite its contributions, *Paper Agent* has limitations that suggest directions for future development:

7.0.0.1 | LaTeX rendering coverage.. The browser-side preview renderer supports a curated subset of LaTeX constructs. Specialized packages (`tikz` for diagrams, `chemfig` for chemical structures, `forest` for linguistic trees) and custom macros are rendered as fallback source text rather than compiled output. Full LaTeX rendering in the browser would require integrating a complete TeX engine via WebAssembly [11] (e.g., SwiftLaTeX [34] or a WebAssembly-compiled TeX Live [36]), which would increase the frontend bundle size by approximately 30–50 MB. A hybrid approach—server-side full rendering with client-side caching for the curated subset—is a promising middle ground that we plan to explore.

7.0.0.2 | Citation database coverage.. Citation verification depends on CrossRef, Semantic Scholar, and OpenAlex. While these databases cover a large fraction of published research, they have gaps in coverage for non-English publications, preprints, conference proceedings in niche venues, and gray literature (technical reports, working papers, theses). Integrating additional sources such as dblp (computer science bibliography), PubMed (biomedical literature), and arXiv (preprints) would improve verification coverage. Citation verification also does not detect fabricated-but-plausible references that match real paper titles but are cited for non-existent claims—a more subtle form of hallucination.

7.0.0.3 | User studies.. We have not conducted formal controlled user studies comparing *Paper Agent* against alternative writing workflows (Overleaf only, ChatGPT-only, manual editing). A rigorous within-subjects or between-subjects experiment measuring writing speed, citation accuracy, compilation success rate, user satisfaction, and perceived control would provide quantitative evidence for the design claims made in this paper. We consider this the most important direction for future work and invite collaboration with HCI researchers.

7.0.0.4 | Collaboration support.. The current system is designed for single-user workflows with optional token-based project sharing. Real-time collaboration similar to Overleaf (multiple simultaneous editors, cursor presence, operational conflict resolution) would require operational transform or CRDT-based synchronization [32], which introduces significant engineering complexity. We have built a preliminary collaboration infrastructure (WebSocket-based merge, token authorization, debounced flush), but full real-time collaborative editing remains future work.

7.0.0.5 | Scalability.. The filesystem-backed project model works well for individual manuscripts but would need enhancement for institutional deployment with hundreds of projects and users. A hybrid model—filesystem storage for manuscript content with a metadata index in SQLite or PostgreSQL for search and discovery—could address this while preserving local-first benefits.

7.0.0.6 | Evaluation scope.. All evaluation was conducted by the development team using the authors' own manuscripts and workflows. This introduces confirmation bias and limits external validity: researchers in other subdisciplines, working styles, or career stages may encounter friction points that the authors did not observe. The system was tested primarily with English-language computer science manuscripts in LaTeX; performance with humanities, social science, or natural science writing conventions—which may rely on different packages, citation styles, or document structures—is unknown. Future evaluation should involve researchers external to the development team and span multiple disciplines.

7.0.0.7 | LLM provider dependency.. *Paper Agent* delegates text generation, review, and polishing to external LLM providers (OpenAI, Anthropic, or locally hosted models via Ollama [23]). The quality and cost of AI assistance therefore depend on the provider's pricing, rate limits, and model capabilities, which change frequently and are outside the system's control. While the multi-provider architecture mitigates single-vendor lock-in, it does not insulate users from API cost escalation or service disruption. Offline operation via local models is possible but requires non-trivial hardware and configuration, placing it beyond the reach of many researchers.

7.0.0.8 | Compile error repair.. When LaTeX compilation fails, the system currently reports errors and warnings in a structured format but does not attempt automatic repair. LLM-based error diagnosis and fix suggestion is a promising direction: the compilation log could be fed to the LLM with the offending source lines, and the AI could suggest corrections. This aligns directly with the document engineering philosophy—treating broken compilation as a system-level error that the system should help resolve, not just report.

7.0.0.9 | **Anti-AI detection..** The system includes a writing-pattern analysis panel (Section 4) that examines surface-level features such as perplexity scores, burstiness, and repetition patterns. We emphasize three critical limitations. First, the effectiveness of AI detection tools against modern LLMs is contested: Liang et al. [19] documented significant rates of both false positives (flagging human-written text as AI-generated) and false negatives (failing to detect AI-generated text), particularly for non-native English writers. Second, this feature must *not* be used as a tool for evading academic-integrity checks or institutional AI-use policies; doing so would undermine scholarly integrity. Third, the writing-pattern metrics are heuristics derived from limited baseline corpora and are discipline-dependent, language-dependent, and model-dependent. We view this feature as strictly experimental and recommend that users treat its output as one data point for self-review—never as proof that text passes or fails AI detection. The interface displays explicit warnings to this effect (Section 4).

8 | Conclusion

This paper has presented *Paper Agent*, a local-first, human-in-the-loop platform that reframes AI-assisted academic writing as a document engineering problem. By grounding AI assistance in a filesystem-backed project model with permission-aware interaction, typed workflow pipelines, and multi-API citation verification, the system addresses three practical problems—context fragmentation, unaudited edits, and unverifiable artifacts—that general-purpose chat interfaces leave unresolved.

The implementation comprises approximately 23,100 lines of application source code across a monorepo with a React/Vite frontend and Fastify/Node.js backend, validated by a 32-file Vitest suite containing 227 automated tests and a Playwright end-to-end scenario.

Our experience building and using *Paper Agent* suggests three broader lessons for the design of AI-assisted writing tools:

1. **Project-level awareness of manuscript structure enables capabilities that text-level tools cannot provide.** When the system understands chapters, citations, and compilation dependencies, it can provide context-aware assistance, verify generated references, and validate edits through compilation—all without additional user effort.
2. **Explicit permission boundaries increase user trust and encourage more exploratory AI use.** Separating AI interaction into Chat, Agent, and Tools modes creates clear mental models and reduces anxiety about unintended changes. Users who were reluctant to use AI in "auto-edit" modes became comfortable with the gated approval workflow.
3. **Local-first architectures provide practical advantages in tool compatibility, data ownership, and configuration flexibility.** Manuscripts remain ordinary git repositories, external tools work naturally, and different projects can use different LLM providers without application changes.

Three directions merit immediate attention. First, controlled user studies comparing document-engineering-aware AI assistance against chat-only or Overleaf-only workflows would provide the quantitative evidence that our current evaluation lacks. Second, scaling the filesystem-backed model to institutional deployments with hundreds of concurrent users requires a metadata indexing layer while preserving local-first benefits. Third, LLM-based compile error repair—feeding compilation logs back to the model for automatic correction—would close the loop between verification and remediation.

Paper Agent is available as open-source software under the MIT license. The source code, documentation, and demonstration projects are accessible at https://github.com/xzktx003/paper_wrighting. We invite contributions from the research software community and look forward to collaborating on controlled user studies to further validate the design claims in this paper.

DATA AVAILABILITY

The software repository includes a minimal redistributable demonstration paper project under `examples/demo-paper/`. No private manuscript data or user data is included. Performance data reported in this paper (citation verification latency, compilation times) were collected from the production APIs and development environment described in Section 5 and are reproducible using the open-source codebase.

ACKNOWLEDGEMENTS

The authors thank the maintainers of the open-source tools and scholarly infrastructure used by *Paper Agent*, including Node.js, React, Vite, Fastify, CodeMirror, KaTeX, TEX Live, CrossRef, Semantic Scholar, OpenAlex, and the Model Context Protocol communities.
We also thank the early adopters who provided valuable feedback during the development of *Paper Agent*, particularly those who tested the permission mode system and pipeline workflows and contributed usability insights that shaped the final design.

FUNDING

The author received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors to support this work.

COMPETING INTERESTS

The author declares no competing interests related to this work.

AUTHOR CONTRIBUTIONS

ZuKang Xu: Conceptualization, Software, Investigation, Writing — original draft, Writing — review & editing, Visualization.

AI ASSISTANCE DISCLOSURE

During the preparation of this manuscript, the author used large language model (LLM) tools (OpenAI GPT-4o and Anthropic Claude) for language polishing, grammar correction, and rephrasing of selected passages. The author has reviewed and edited all AI-generated content and takes full responsibility for the final manuscript. These tools were not used for generating research ideas, experimental designs, data analysis, or core argumentation. No AI tool is listed as an author.

References

[1] Anthropic. The claude model family. *Anthropic Technical Report*, 2024. URL https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3_Addendum.pdf.
[2] Anthropic. Model context protocol, 2024. URL https://modelcontextprotocol.io/.
[3] Cactus Communications. Paperpal: AI academic writing assistant, 2024. URL https://paperpal.com/.
[4] CrewAI. CrewAI: Framework for orchestrating role-playing AI agents, 2024. URL https://www.crewai.com/.
[5] Crossref. Crossref REST API, 2024. URL https://api.crossref.org/.
[6] Dynalist Inc. Obsidian: A knowledge base that works on top of a local folder of markdown files, 2024. URL https://obsidian.md/.
[7] Fastify Team. Fastify: Fast and low overhead web framework for Node.js, 2024. URL https://fastify.dev/.
[8] GitHub, Inc. GitHub Copilot: AI pair programmer, 2024. URL https://github.com/features/copilot.
[9] GitHub, Inc. GitHub Actions: Automate your workflow from idea to production, 2024. URL https://github.com/features/actions.
[10] Grammarly. Grammarly: AI-powered writing assistant, 2024. URL https://www.grammarly.com/.
[11] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017. doi: 10.1145/3062341.3062363.
[12] Marijn Haverbeke. Codemirror: Extensible code editor, 2024. URL https://codemirror.net/.
[13] Ian Hickson. Server-sent events. W3C Recommendation, 2015. URL https://html.spec.whatwg.org/multipage/server-sent-events.html.
[14] JSON-RPC Working Group. JSON-RPC 2.0 specification, 2010. URL https://www.jsonrpc.org/specification.
[15] KaTeX Team. KaTeX: The fastest math typesetting library for the web, 2024. URL https://katex.org/.

[16] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, et al. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90, 2016. doi: 10.3233/978-1-61499-649-1-87.
[17] LangChain. LangChain: Build context-aware reasoning applications, 2024. URL https://www.langchain.com/.
[18] LangChain. LangGraph: Building stateful, multi-actor applications with LLMs, 2024. URL https://langchain-ai.github.io/langgraph/.
[19] Weixin Liang, Yaohui Zhang, Zhengliang Wu, Haley Lepp, Wenlong Ji, Xuandong Zhao, Hancheng Cao, Sheng Liu, Siyu He, Zhi Huang, et al. Mapping the increasing use of LLMs in scientific papers. *arXiv preprint arXiv:2404.01268*, 2024. doi: 10.48550/arXiv.2404.01268.
[20] Nicholas Marriott. tmux: Terminal multiplexer, 2024. URL https://github.com/tmux/tmux.
[21] Meta Open Source. React: A javascript library for building user interfaces, 2024. URL https://react.dev/.
[22] Microsoft. Microsoft copilot, 2024. URL https://www.microsoft.com/en-us/copilot/.
[23] Ollama. Ollama: Run LLMs locally, 2024. URL https://ollama.com/.
[24] OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. doi: 10.48550/arXiv.2410.21276.
[25] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyam Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. doi: 10.48550/arXiv.2303.08774.
[26] OpenAlex. OpenAlex: An open catalog of the global research system, 2024. URL https://openalex.org/.
[27] Overleaf. Overleaf: The online LaTeX editor, 2024. URL https://www.overleaf.com/.
[28] Overleaf. Overleaf AI features, 2024. URL https://docs.overleaf.com/integrations-and-add-ons/ai-features.
[29] Posit. Quarto: An open-source scientific and technical publishing system, 2024. URL https://quarto.org/.
[30] RStudio and Posit. R Markdown: Dynamic documents for R, 2024. URL https://rmarkdown.rstudio.com/.
[31] Semantic Scholar. Semantic scholar API, 2024. URL https://www.semanticscholar.org/product/api.
[32] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. A comprehensive study of convergent and commutative replicated data types. Technical Report RR-7506, INRIA, 2011. URL https://inria.hal.science/inria-00555588.
[33] Software Freedom Conservancy. Git: Distributed version control system, 2024. URL https://git-scm.com/.
[34] SwiftLaTeX Project. SwiftLaTeX: A LaTeX renderer in WebAssembly, 2023. URL https://www.swiftlatex.com/.
[35] Tectonic Project. Tectonic: A modernized, complete, self-contained TeX/LaTeX engine, 2024. URL https://tectonic-typesetting.github.io/.
[36] TeX Users Group. TeX Live: A comprehensive distribution of the TEX typesetting system, 2024. URL https://www.tug.org/texlive/.
[37] TeXstudio Team. TeXstudio: LaTeX editor, 2024. URL https://www.texstudio.org/.
[38] Trinka AI. Trinka: AI-powered technical writing assistant, 2024. URL https://www.trinka.ai/.
[39] Vite Team. Vite: Next generation frontend tooling, 2024. URL https://vitejs.dev/.
[40] Vitest Team. Vitest: Next generation testing framework, 2024. URL https://vitest.dev/.
[41] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2014. doi: 10.1007/978-3-642-29044-2.
[42] Writefull. Writefull: AI-based academic writing support, 2024. URL https://www.writefull.com/.
[43] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. doi: 10.48550/arXiv.2308.08155.
[44] Xingjiao Wu, Luwei Xiao, Yixuan Sun, Junhui Zhang, Tianlong Ma, and Liang He. A survey of human-in-the-loop machine learning. *Future Generation Computer Systems*, 135:364–381, 2022. doi: 10.1016/j.future.2022.05.014.
[45] James Yu. LaTeX Workshop: Extension for VS Code, 2024. URL https://github.com/James-Yu/LaTeX-Workshop.